EF Core: HAVING with GroupBy()

Filtering groups after aggregation | Concise lesson summary

Core idea

HAVING filters grouped results after grouping and aggregation. In EF Core, a Where() applied to a grouped query—such as GroupBy(...).Where(g => g.Count() > 2)—can translate to SQL HAVING when the query shape is translatable.

Mental anchor: WHERE filters rows before grouping. HAVING filters groups after grouping.

WHERE vs HAVING

Clause	Filters	Example
WHERE	Individual rows before grouping.	`WHERE Status = 'Active'`
HAVING	Groups after grouping/aggregation.	`HAVING COUNT(*) > 2`

Example: statuses with more than 2 students

StudentsPerStatus.cs — C# • EF Core

```
1  var query = context.Students
2  .GroupBy(s => s.Status)
3  .Where(g => g.Count() > 2)
4  .Select(g => new
5  {
6  Status = g.Key,
7  TotalStudents = g.Count()
8  })
9  .OrderBy(x => x.Status);
10
11 var report = query.ToList();
```

g.Key is the group key (the status). The count filters groups and is also returned in the report.

Typical SQL generated

Having.sql — C# • EF Core

```
1  SELECT [s].[Status], COUNT(*) AS [TotalStudents]
2  FROM [Students] AS [s]
3  GROUP BY [s].[Status]
4  HAVING COUNT(*) > 2
5  ORDER BY [s].[Status];
```

Why database-side filtering matters

The database can group rows and filter groups before returning the report. This avoids loading all raw rows and performing grouping/filtering in application memory, which can be expensive for large tables.

AvoidThis.cs C# • EF Core

```
var students = context.Students.ToList();
var report = students
.GroupBy(s => s.Status)
.Where(g => g.Count() > 2);

// Prefer a translatable database query
var report = context.Students
.GroupBy(s => s.Status)
.Where(g => g.Count() > 2)
.Select(g => new { Status = g.Key, Count = g.Count() })
.ToList();
```

The first version loads rows before grouping. The second lets EF Core translate the grouped report when the query shape is supported.

More reporting patterns

ReportingPatterns.cs C# • EF Core

```
// Courses with more than 10 enrollments
var popularCourses = context.Enrollments
.GroupBy(e => e.CourseId)
.Where(g => g.Count() > 10)
.Select(g => new { CourseId = g.Key, Count = g.Count() });

// Instructors teaching more than 3 courses
var busyInstructors = context.Courses
.GroupBy(c => c.InstructorId)
.Where(g => g.Count() > 3)
.Select(g => new { InstructorId = g.Key, Count = g.Count() });
```

SQL preview and runtime logging

ToQueryString() previews the SQL shape for an IQueryable query; it does not execute it. Runtime logging configured through LogTo() shows the SQL actually executed when the query is materialized or otherwise executed.

Practical checklist

- Use Where() before GroupBy() to filter individual rows.
- Use Where() after GroupBy() to filter groups; EF Core may translate this to HAVING.
- Project groups into report-shaped results with Select(), using g.Key and aggregates such as Count() or Average().
- Keep report filtering in the database when the query is translatable.
- HAVING is useful for dashboards, KPI reports, threshold alerts, and admin summaries.

Remember: WHERE decides which rows participate. HAVING decides which summarized groups remain in the final report.